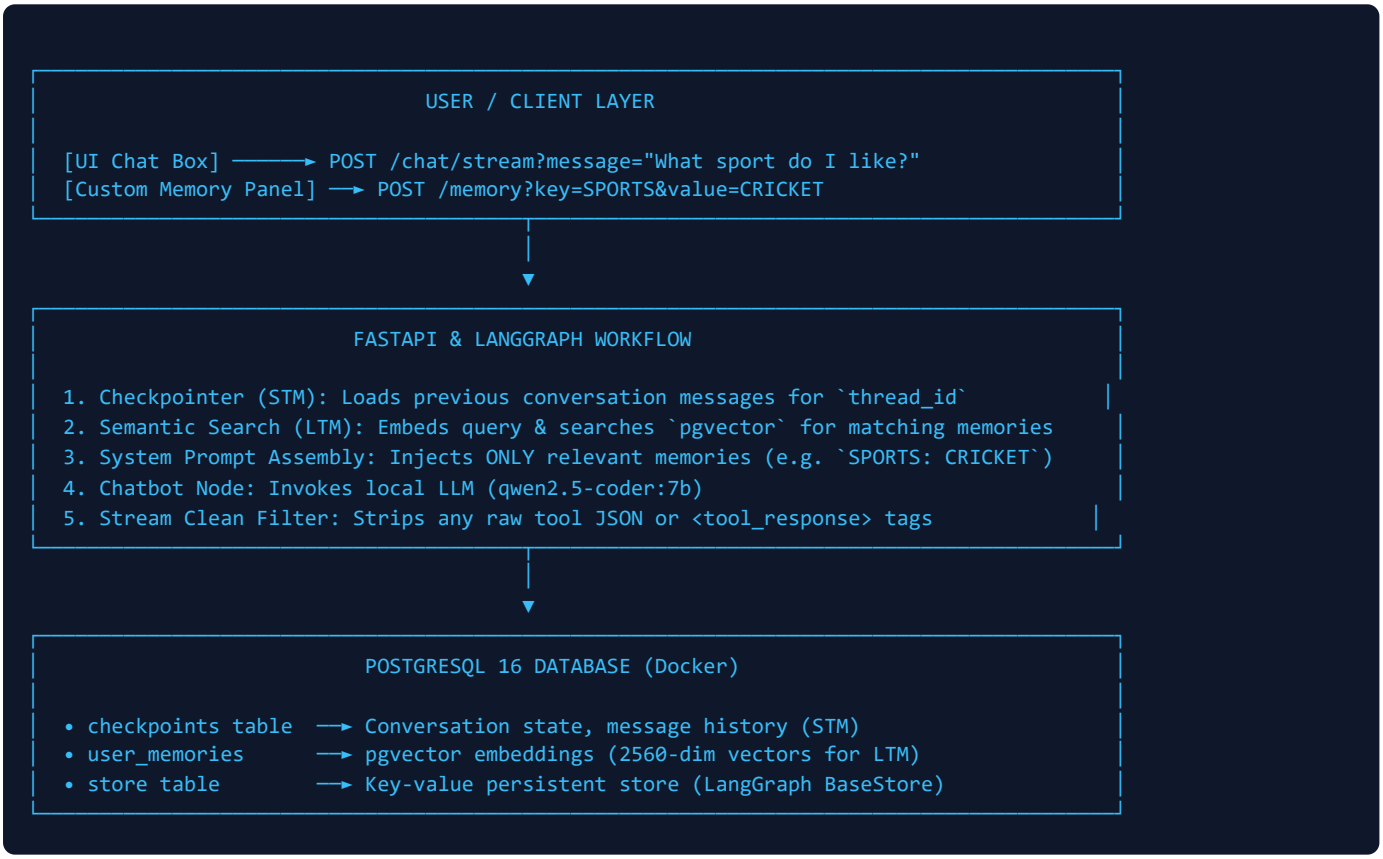
🧠 AI Coding Agent: How Memory Works (Deep Dive)

Comprehensive Architecture Guide (Code-level explanations, Flow Diagrams & Hinglish Insights)

> 📌 **System Overview:** AI Coding Agent mein human brain ki tarah **2 alag Memory Systems** kaam karte hain:
>
> - STM **Short-Term Memory (Chat History):** Current conversation thread ka context yaad rakhna.
> - LTM **Long-Term Memory (Semantic Vector Store):** Permanent preferences, sports, coding language, and personal rules ko har thread mein preserve karna without prompt bloat.

---

1. High-Level Architectural Flow



---

2. Short-Term Memory (STM) in Depth

Key Question: "Ek chat mein AI ko purani baatein kaise yaad rehti hain?"

Jab aap chat mein bolte ho "*Mera naam Rahul hai*" aur 2 minute baad puchte ho "*Mera naam kya hai?*", yeh kaam **Short-Term Memory (STM)** ka hota hai.

Code Implementation: `backend/db/connection.py`

```
from langgraph.checkpoint.postgres import PostgresSaver
from psycopg_pool import ConnectionPool

pool = ConnectionPool(conninfo=DB_URI, max_size=20, kwargs={"autocommit": True})

# LangGraph Checkpointer - persists conversation turns per thread_id
checkpointer = PostgresSaver(pool)
checkpointer.setup()
```

Kaise Kaam Karta Hai (Step-by-Step):

1. **Thread Isolation:** Har conversation ka ek unique `thread_id` hota hai (jaise: `session_oow1nt`).
2. **State Serialization:** Har question aur AI answer PostgreSQL ke `checkpoints` table mein serialize ho kar save hota hai.
3. **Next Message Context:** Jab user naya message bhejta hai, LangGraph `thread_id` se purani saari messages list load karke LLM ko pass karta hai.
4. **Boundary (Limit):** Jaise hi aap **"+ New Conversation"** dabate ho, naya `thread_id` banta hai — isliye purani chat ka data naye thread mein leak nahi hota.

3. Long-Term Memory (LTM) & Semantic Vector Search

The Real Challenge: "Prompt Bloat Problem"

> Agar hum database ki **saari 500 ya 1000 memories** ko har chat mein prompt ke andar daal denge, toh:
>
> - Prompt bohot bada (bloated) ho jayega.
> - LLM tokens waste honge aur inference slow ho jayega.
> - Irrelevant memories (jaise food preferences) coding tasks ko confuse kar dengi.
>
> **Solution:** `Semantic Memory Search` via **pgvector**!

Code Implementation: `backend/rag/vector_db.py`

```
from langchain_ollama import OllamaEmbeddings
from langchain_postgres import PGVector

# 2560-dimension embeddings via Ollama
embeddings = OllamaEmbeddings(model="qwen3-embedding:4b")

# pgvector collection specifically for User Memories
memory_vector_db = PGVector(
    embeddings=embeddings,
    collection_name="user_memories",
    connection=DB_URI,
    use_jsonb=True
)

def save_semantic_memory(key: str, value: str, user_id: str = "1"):
    from langchain_core.documents import Document
    doc = Document(
        page_content=f"{key}: {value}",
        metadata={"key": key, "user_id": user_id}
    )
    memory_vector_db.add_documents([doc])
```

Smart Retrieval Logic: `backend/graph/nodes.py`

Jab user message bhejta hai, hum user query ka vector nikaal kar pgvector se **Cosine Distance** calculate karte hain:

```
def chatbot(state: MyState):
    memory_context = ""
    try:
        from rag.vector_db import memory_vector_db
        from langchain_core.messages import HumanMessage

        last_user_query = [m.content for m in reversed(state.messages) if isinstance(m, HumanMessage)

        # Semantic Similarity Search in pgvector
        results = memory_vector_db.similarity_search_with_score(last_user_query, k=4)
        selected = []
        for doc, score in results:
```

```
            # Distance < 0.50 means a strong semantic match
            if score < 0.50:
                selected.append(f"- {doc.page_content}")
                if len(selected) ≥ 2: break

        if selected:
            memory_context = "\n\n[RELEVANT USER MEMORY]:\n" + "\n".join(selected)
    except Exception:
        pass

    system_msg = SystemMessage(content=SYSTEM_PROMPT + memory_context)
    return {"messages": [llm_with_tools.invoke([system_msg] + list(state.messages))]}
```

🔥 **Mathematical Filtering in Action:**

User Query	Database Memory	Cosine Distance	Action
"What sport do I like?"	`SPORTS: CRICKET`	**0.4384**	✅ Injected into prompt!
"What sport do I like?"	`evening_food: momos`	0.8210	❌ Rejected (Irrelevant)
"What sport do I like?"	`COADING LANGUAGE: PYTHON`	0.7950	❌ Rejected (Irrelevant)
"How to write quicksort?"	`SPORTS: CRICKET`	0.6120	❌ Rejected (Prompt remains clean)

4. Clean Streaming Filter (Preventing Tool JSON Leaks)

Problem: "Screen par `<tool_response>` ya JSON kyu aa raha tha?"

Local Ollama models (jaise `qwen2.5-coder:7b`) jab background tools call karte hain, toh wo JSON syntax ya XML tags generate karte hain. Pehle streaming endpoint (`stream_mode="messages"`) in tokens ko bina filter kiye frontend par stream kar deta tha.

Solution: `backend/routers/chat.py`

```
def clean_response_text(text: str) → str:
    # Strip <tool_response> ... </tool_response>
    cleaned = re.sub(r'<tool_response>[\s\S]*?</tool_response>', '', text, flags=re.IGNORECASE).stri
    # Strip <tool_call> ... </tool_call>
    cleaned = re.sub(r'<tool_call>[\s\S]*?</tool_call>', '', cleaned, flags=re.IGNORECASE).strip()
    # Strip raw JSON code blocks and inline JSON
    cleaned = re.sub(r'```(?:json)?\s*\{[\s\S]*?\}\s*```', '', cleaned).strip()
    cleaned = re.sub(r'\{\s*"name"\s*:\s*"[^"]+".*?\}\s*\}?', '', cleaned).strip()
    return cleaned if cleaned else text
```

Humne `stream_mode="updates"` use kiya jo ensures karta hai ki tool executions behind-the-scenes finish hone ke baad hi final response token-by-token frontend ko stream ho.

---

5. Summary: STM vs LTM Comparison Matrix

Dimension	Short-Term Memory (STM)	Long-Term Memory (LTM)
Core Goal	Current chat session ki continuous flow	Permanent user facts, habits & preferences
Persistence Level	Scoped to individual `thread_id`	Global across all threads for `user_id`
Database Engine	`PostgresSaver` (PostgreSQL Table: `checkpoints`)	`PGVector` (PostgreSQL Collection: `user_memories`)
Retrieval Mechanism	Chronological full state history	**Semantic Cosine Similarity Search (k=2)**
Scaling Safety	Trimmed via message window / summarizer	**Immune to bloat:** 1000+ memories mein se sirf 1-2 load hongi
UI Touchpoint	Left Sidebar: "Threads & History"	Left Sidebar: "Custom Memory" Collapsible Panel

---

6. File Structure Reference

- `backend/db/connection.py` —— PostgreSQL Pool & PostgresSaver Checkpointer setup

- `backend/rag/vector_db.py` — PGVector Codebase & User Memories collections + Embeddings
- `backend/graph/nodes.py` — Chatbot node with Semantic Vector Memory retrieval
- `backend/graph/tools.py` — Tools: `save_memory`, `get_memory`, `list_memories`
- `backend/routers/chat.py` — Clean streaming endpoint without internal tool leaks
- `backend/routers/memory.py` — REST endpoints to save/read permanent user memories